The Journal of Open Source Software

POUNCE: an AI-first, pure-Rust interior-point optimizer for nonlinear, convex and complementarity-constrained problems

John R. Kitchin [1¶], **David E. Bernal Neira**[2], **Johannes P. Dürholt**[3], **Frederik Geth**[4], **Devin Griffith**[5], **and Germán Heim**[6]

1 Department of Chemical Engineering, Carnegie Mellon University, Pittsburgh, PA, United States **2** Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN, United States **3** Evonik Operations GmbH, Germany **4** The University of Queensland, Australia **5** TODO (affiliation to confirm) **6** Universidad Nacional del Sur, Bahía Blanca, Argentina ¶ Corresponding author

DOI: N/A

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: 01 January 1970

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Many problems in engineering and science are optimization problems: choose the operating conditions of a chemical plant, fit a model to data, or set the power flows on a grid, subject to physical laws written as equations and limits written as inequalities. POUNCE solves such problems. Its core is a port of Ipopt (Wächter & Biegler, 2006), the interior-point solver most widely used for large, sparse, smooth nonlinear programs. Around that core it adds engines for convex quadratic and conic programs, active-set QP and SQP, parametric sensitivity (Pirnay et al., 2012), multistart searches for several local minima, and problems with complementarity constraints (MPCCs), in which one of two quantities must be zero at every pair.

POUNCE is written entirely in Rust, including its default sparse symmetric-indefinite linear solver, FERAL. It has no Fortran, BLAS or HSL dependency, so it builds from `cargo`, installs from `pip` on Linux, macOS and Windows, and runs in a web browser through WebAssembly and Pyodide. It reads AMPL `.nl` files and is available from Python (a `scipy.optimize.minimize`-compatible API and a `Problem` API), Pyomo (Bynum et al., 2021), CasADi (Andersson et al., 2019), Julia through JuMP and MathOptInterface (Legat et al., 2022; Lubin et al., 2023), GAMS, a C API, and a container image.

POUNCE was built AI-first. The human authors set the scope, the architecture and the standards of evidence. Most of the code, tests and documentation were written by an AI coding agent working under those standards. We describe that process, and how its output was checked, in **AI usage disclosure**.

Statement of need

Ipopt is the default local NLP solver in most algebraic modeling systems. Using it well has a practical cost. Good performance needs a licensed HSL linear solver (Duff, 2004) or a separately built MUMPS (Amestoy et al., 2001), and the Fortran and C++ toolchain makes it hard to ship in a Python wheel or a browser. Teaching, web demonstrations, and research codes that people install with a single `pip install` all run into this cost. POUNCE removes it. It keeps Ipopt's algorithm, option names and console output, so existing models, option files and experience carry over unchanged.

The second need is for solver features that are usually separate programs. POUNCE routes each model to the most specialized engine it supports. A convex QP goes to a homogeneous self-dual interior point (Ye et al., 1994), a general NLP to the Ipopt port, and a model with

declared complementarity pairs to a driver that runs Scholtes regularization (Scholtes, 2001) and an exact product formulation, then certifies the answer as strongly or weakly stationary. Sensitivity of the solution to parameters, a multistart search for distinct local minima, and a machine-readable JSON solve report are built in, not add-ons.

The intended users are researchers and students in process systems engineering, power systems and optimal control. They write models in Pyomo, CasADi, JuMP or AMPL and need a solver that installs anywhere and explains its answers.

State of the field

Ipopt (Wächter & Biegler, 2006) is the reference implementation that POUNCE follows. MadNLP (Shin et al., 2024) is a Julia interior-point solver that targets GPUs and condensed-space methods. Uno (Vanaret & Montoison, 2026) is a composable C++ framework that can reproduce several existing solver designs. Clarabel (Goulart & Chen, 2024) is a Rust conic solver for convex problems only. None of these is a dependency-free, drop-in replacement for Ipopt that can also be shipped to WebAssembly.

The choice between building and contributing was made in favour of a port. The goal was Ipopt's behaviour without its toolchain. That cannot be added to Ipopt itself, and it is a different goal from MadNLP's or Uno's. The port follows upstream closely enough to compare iterate by iterate. Where POUNCE departs from upstream, it does so behind an option whose documented value restores upstream behaviour. Examples are the cap on slack-row perturbations and the bound on tiny-step dual stalls.

Software design

POUNCE is a Cargo workspace of 25 crates. They separate the parts a reviewer would want to check on their own: the NLP algorithm (`pounce-algorithm`, `pounce-nlp`, `pounce-restoration`), linear algebra and linear solvers (`pounce-linalg`, `pounce-linsol`, `pounce-feral`, optional `pounce-hsl`), the convex and active-set engines (`pounce-convex`, `pounce-qp`), presolve, sensitivity, multistart, the `.nl` reader, and the front ends (CLI, Python, C, WebAssembly).

Three design decisions follow from the research setting.

1. **Upstream parity is the default.** The port keeps Ipopt's data flow and option semantics. A fix that changes the trajectory is measured against upstream and against the previous release before it merges.
2. **One entry point, routing to many engines.** At the default `solver_selection=auto`, POUNCE classifies each model and sends it to the most specialized engine available. Every report records which engine answered, so a routing change shows up in the record even when the status and objective do not move.
3. **Answers carry their own evidence.** Each solve can emit a versioned JSON report. It contains residuals, the iterations spent in every re-solve and fallback, and, for complementarity models, the stationarity certificate and how complementary the returned point is. A companion tool, `pounce-studio`, diagnoses a solve from that report.

Research impact statement

POUNCE has been public since May 2026 and has had 15 releases. `pounce-solver` has been downloaded about 42,600 times from PyPI since its first release on 25 May 2026, excluding mirrors (about 55,000 including them). `pyomo-pounce` has been downloaded about 14,500 times. Hans Mittelmann's public benchmark of AMPL NLP solvers includes POUNCE (Mittelmann, n.d.).

POUNCE is in research use outside its authors' group:

- Opyrability, a process-operability package (Alves et al., 2024), uses POUNCE as its default solver.
- SiNDAE, which trains neural differential-algebraic equations by the simultaneous approach (Lueg & others, 2025), runs its whole pipeline on POUNCE.
- oximo, a Rust algebraic modeling library, has a POUNCE backend, and tellegen runs live interactive power-flow analysis on it.
- The Julia interface was contributed by power-systems researchers and is validated against an optimal power flow adapter.

On the release benchmark (1,345 problems from the CUTE/Vanderbei AMPL collection, Mittelmann's NLP set, Maros–Mészáros QPs (Maros & Mészáros, 1999), LP collections, and application suites), POUNCE solves 1,309 to strict optimality, against 1,260 for Ipopt 3.14 with MA57 under the same 300 s limit. On MacMPEC, it reaches the best known objective on 158 of 178 problems and leaves none unsolved. Performance profiles (Dolan & Moré, 2002) and the full per-problem tables are in the repository.

AI usage disclosure

POUNCE was developed with Anthropic's Claude, through the Claude Code agent. Of the 3,711 commits on the main branch at version 0.16.0, 2,712 (73%) are authored or co-authored by Claude. AI-written work includes the Rust port, tests, benchmark harnesses, documentation, notebooks and a first draft of this paper. The human authors decided what to build, chose the architecture (a faithful port first, extensions behind options, engine routing, the evidence-carrying report) and set how correctness is shown. Every change was reviewed by a human before it merged.

The verification regime is designed for code that is cheap to produce and easy to get subtly wrong:

- Changes are checked against independent oracles, not only against tests written alongside them: upstream Ipopt's iterates, published best-known objectives, MILP-certified global optima, and recomputed KKT and complementarity residuals.
- Any change that can alter the solver's path is run through a fixture sweep on two Hessian modes and two CPU architectures, and every moved line must be explained before merge. The sweep exists because a regression that keeps the right answer but takes nearly four times the iterations passed a status-and-objective test suite and shipped.
- Defects that tests missed are written up as post-mortems in `dev-notes/` and distilled into the agent's standing instructions (`CLAUDE.md`), so later work inherits them.
- An adversarial test harness poses problems with known solutions to every engine and reports wrong answers.

The authors take full responsibility for the software and this paper.

Acknowledgements

POUNCE's NLP core is a port of Ipopt, by Andreas Wächter, Lorenz T. Biegler and the COIN-OR community. It succeeds ripopt (Kitchin, 2026).

References

Alves, V., Dinh, S., Kitchin, J. R., Gazzaneo, V., Carrasco, J. C., & Lima, F. V. (2024). Opyrability: A Python package for process operability analysis. *Journal of Open Source Software*, *9*, 5966. https://doi.org/10.21105/joss.05966

Amestoy, P. R., Duff, I. S., L'Excellent, J.-Y., & Koster, J. (2001). A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, *23*(1), 15–41. https://doi.org/10.1137/S0895479899358194

Andersson, J. A. E., Gillis, J., Horn, G., Rawlings, J. B., & Diehl, M. (2019). CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, *11*(1), 1–36. https://doi.org/10.1007/s12532-018-0139-4

Bynum, M. L., Hackebeil, G. A., Hart, W. E., Laird, C. D., Nicholson, B. L., Siirola, J. D., Watson, J.-P., & Woodruff, D. L. (2021). *Pyomo – optimization modeling in python* (3rd ed., Vol. 67). Springer. https://doi.org/10.1007/978-3-030-68928-5

Dolan, E. D., & Moré, J. J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, *91*(2), 201–213. https://doi.org/10.1007/s101070100263

Duff, I. S. (2004). MA57 – a code for the solution of sparse symmetric definite and indefinite systems. *ACM Transactions on Mathematical Software*, *30*(2), 118–144. https://doi.org/10.1145/992200.992202

Goulart, P. J., & Chen, Y. (2024). *Clarabel: An interior-point solver for conic programs with quadratic objectives*. https://doi.org/10.48550/arXiv.2405.12762

Kitchin, J. R. (2026). *Ripopt: A memory-safe interior point optimizer in Rust*. Zenodo. https://doi.org/10.5281/zenodo.19542664

Legat, B., Dowson, O., Dias Garcia, J., & Lubin, M. (2022). MathOptInterface: A data structure for mathematical optimization problems. *INFORMS Journal on Computing*, *34*(2), 672–689. https://doi.org/10.1287/ijoc.2021.1067

Lubin, M., Dowson, O., Dias Garcia, J., Huchette, J., Legat, B., & Vielma, J. P. (2023). JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, *15*, 581–589. https://doi.org/10.1007/s12532-023-00239-3

Lueg, L. R., & others. (2025). *A simultaneous approach for training neural differential-algebraic systems of equations*. https://doi.org/10.48550/arXiv.2504.04665

Maros, I., & Mészáros, C. (1999). A repository of convex quadratic programming problems. *Optimization Methods and Software*, *11*(1–4), 671–681. https://doi.org/10.1080/10556789908805768

Mittelmann, H. D. (n.d.). *Benchmarks for optimization software: AMPL-NLP benchmark*. https://plato.asu.edu/ftp/ampl-nlp.html.

Pirnay, H., López-Negrete, R., & Biegler, L. T. (2012). Optimal sensitivity based on IPOPT. *Mathematical Programming Computation*, *4*(4), 307–331. https://doi.org/10.1007/s12532-012-0043-2

Scholtes, S. (2001). Convergence properties of a regularization scheme for mathematical programs with complementarity constraints. *SIAM Journal on Optimization*, *11*(4), 918–936. https://doi.org/10.1137/S1052623499361233

Shin, S., Anitescu, M., & Pacaud, F. (2024). Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, *236*, 110651. https://doi.org/10.1016/j.epsr.2024.110651

Vanaret, C., & Montoison, A. (2026). Uno: A composable framework for nonlinearly constrained optimization. *Journal of Open Source Software*, *11*(123), 10229. https://doi.org/10.21105/joss.10229

Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*,

106(1), 25–57. https://doi.org/10.1007/s10107-004-0559-y

Ye, Y., Todd, M. J., & Mizuno, S. (1994). An $O(\sqrt{n}L)$-iteration homogeneous and self-dual linear programming algorithm. *Mathematics of Operations Research*, *19*(1), 53–67. https://doi.org/10.1287/moor.19.1.53